

Aress Frontend Project - Ultimate Technical Documentation

Table of Contents

Project Overview

Key Features

Architecture & Technology Stack

Core Technologies

Development Tools

Infrastructure

Project Structure

OpenAPI Library & Authentication Deep Dive

OpenAPI Architecture

Authentication Methods

Rendering Strategies

Next.js 15 App Router Architecture

Design System - Complete Component Analysis

Overview

Component Architecture

All Components - Detailed Implementation

Complete Component Inventory

Detailed Component Analysis

Complex Components Analysis

1. VideoPlayer Component - Advanced Implementation

2. GeneralTable Component - Data Management

3. Form Components - Validation & UX

4. Dialog/Modal System

State Management Patterns

React Query Configuration

State Management Strategies

Performance Optimization Strategies

1. Component-Level Optimizations

2. Data Fetching Optimizations

3. Bundle Optimization

Applications - Every Page Analysis

FE-App (B2B Application) - Detailed Page Breakdown

B2C-App (Consumer Application) - Complete Page Analysis

Development Setup

Prerequisites

Installation Steps

Development Commands

API Integration

OpenAPI/Swagger Integration

Testing Strategy

Testing Pyramid

Deployment

Build Process

Deployment Targets

Best Practices & Guidelines
Component Development
Performance Optimization
Security Considerations
Troubleshooting
Common Issues

Aress Frontend Project - Ultimate Technical Documentation

Table of Contents

1. [Project Overview](#)
 2. [Architecture & Technology Stack](#)
 3. [Project Structure](#)
 4. [OpenAPI Library & Authentication Deep Dive](#)
 5. [Rendering Strategies](#)
 6. [Design System - Complete Component Analysis](#)
 7. [All Components - Detailed Implementation](#)
 8. [Applications - Every Page Analysis](#)
 9. [State Management Patterns](#)
 10. [Performance Optimization Strategies](#)
 11. [Development Setup](#)
 12. [API Integration](#)
 13. [Testing Strategy](#)
 14. [Deployment](#)
-

Project Overview

The **Aress Frontend Project** is a sophisticated enterprise-grade monorepo built with Nx, designed to support multiple frontend applications with a shared design system. The project emphasizes code reusability, maintainability, and developer experience through modern tooling and best practices.

Key Features

- **Monorepo Architecture:** Nx-powered workspace for efficient code sharing
- **Comprehensive Design System:** 60+ reusable UI components
- **Multiple Applications:** B2B and B2C applications with shared infrastructure
- **Modern Tooling:** TypeScript, Tailwind CSS, Storybook, Vitest

- **API-First Approach:** OpenAPI/Swagger integration with code generation
 - **Production Ready:** Docker support, CI/CD workflows, and quality gates
-

Architecture & Technology Stack

Core Technologies

- **Framework:** React 18+ with Next.js 15
- **Language:** TypeScript 5+
- **Styling:** Tailwind CSS with custom design tokens
- **Build System:** Nx 21.3.0 with Vite/Webpack
- **Package Manager:** pnpm (workspace support)

Development Tools

- **Component Development:** Storybook 9.0
- **Testing:** Vitest + Jest + Playwright
- **Code Quality:** ESLint, Prettier, Husky, Commitlint
- **Icons:** Iconify with Lucide icon set
- **Documentation:** Storybook with auto-docs

Infrastructure

- **Containerization:** Docker with multi-stage builds
 - **Deployment:** Vercel integration
 - **API Integration:** OpenAPI code generation
 - **State Management:** React Query (TanStack Query)
-

Project Structure

```
ares-frontend/
├── apps/                                # Applications
│   ├── fe-app/                         # Main B2B application (Next.js)
│   ├── b2c-app/                       # B2C application (Next.js)
│   ├── fe-app-e2e/                   # E2E tests for fe-app
│   └── renderer/                     # Rendering service
├── libs/                              # Shared libraries
│   ├── design-system/               # UI component library
│   ├── openapi/                    # Generated API clients
│   └── shared/                      # Shared utilities and hooks
├── scripts/                          # Build and utility scripts
└── shared/                           # Global shared resources
```

└─ .storybook/	# Global Storybook configuration
└─ docker-compose.yaml	# Development environment
└─ nx.json	# Nx workspace configuration
└─ package.json	# Root package configuration

OpenAPI Library & Authentication Deep Dive

OpenAPI Architecture

The project uses a sophisticated OpenAPI integration system that automatically generates TypeScript clients and React Query hooks from Swagger specifications.

Core OpenAPI Configuration

Location: `libs/openapi/src/requests/core/OpenAPI.ts`

```
export type OpenAPIConfig = {
  BASE: string; // API base URL
  CREDENTIALS: 'include' | 'omit' | 'same-origin';
  ENCODE_PATH?: ((path: string) => string) | undefined;
  HEADERS?: Headers | Resolver<Headers> | undefined;
  PASSWORD?: string | Resolver<string> | undefined;
  TOKEN?: string | Resolver<string> | undefined;
  USERNAME?: string | Resolver<string> | undefined;
  VERSION: string;
  WITH_CREDENTIALS: boolean;
  interceptors: {
    request: Interceptors<RequestInit>;
    response: Interceptors<Response>;
  };
};
```

Authentication Strategy

Dual Token Management System:

1. Client-Side Token Storage (localStorage):

```
TOKEN: async () =>
  typeof window !== 'undefined'
    ? localStorage.getItem('access_token') || ''
    : '',
```

2. Server-Side Token Storage (HTTP-only cookies):

```
// In login API route
response.cookies.set('access_token', data.access_token, {
  httpOnly: true,
  secure: false,
  sameSite: 'lax',
  path: '/',
  maxAge: 60 * 60 * 24 * 7, // 1 week
});
```

Request/Response Interceptors

Advanced Interceptor System:

```
export class Interceptors<T> {
  _fns: Middleware<T>[];

  constructor() {
    this._fns = [];
  }

  eject(fn: Middleware<T>): void {
    const index = this._fns.indexOf(fn);
    if (index !== -1) {
      this._fns = [...this._fns.slice(0, index),
        ...this._fns.slice(index + 1)];
    }
  }

  use(fn: Middleware<T>): void {
    this._fns = [...this._fns, fn];
  }
}
```

Usage Examples: - **Request Interceptor:** Add authentication headers, logging, request transformation - **Response Interceptor:** Handle errors, transform responses, refresh tokens

Generated API Structure

Auto-Generated Files: - schemas.gen.ts (130KB) - Complete type definitions - services.gen.ts (53KB) - Service layer implementations - types.gen.ts (95KB) - TypeScript interfaces - queries/ - React Query hooks for each endpoint

Code Generation Workflow

```
# 1. Merge multiple Swagger files
pnpm merge:swagger
```

2. Generate TypeScript API client

pnpm generate:sdk

3. Generate React Query hooks

pnpm generate:query

4. Complete workflow

pnpm output:swagger

Authentication Methods

1. Login Flow Implementation

Frontend Login Process (apps/fe-

app/app/(auth)/login/_components/FormWrapper.tsx):

```
const handleLogin = async (values: LoginFormValues) => {
  try {
    const res = await fetch('/api/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        username: values.username,
        password: values.password,
        captcha: values.captcha ?? '',
        captchaUid: typeof values.captchaUid === 'number' ?
          values.captchaUid : 0,
      }),
    });

    const data = await res.json();
    if (res.ok && data.success) {
      // Store token in localStorage for OpenAPI client usage
      if (data.access_token) {
        localStorage.setItem('access_token', data.access_token);
      }
      router.push('/');
    }
  } catch (error) {
    // Error handling with toast notifications
  }
};
```

Backend Login API (apps/fe-app/app/api/login/route.ts):

```
export async function POST(req: NextRequest) {
  const { username, password, captcha, captchaUid } = await
    req.json();
```

```

const params = new URLSearchParams({ username, password });
const url = `${process.env.NEXT_PUBLIC_API_URL}/users/login?
  captchaUid=${captchaUid}&captcha=${encodeURIComponent(captcha)}`;

const res = await fetch(url, {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded'
  },
  body: params.toString(),
});

const data = await res.json();

if (res.ok && data.access_token) {
  const response = NextResponse.json({
    success: true,
    access_token: data.access_token,
  });

  // Set secure HTTP-only cookie
  response.cookies.set('access_token', data.access_token, {
    httpOnly: true,
    secure: false,
    sameSite: 'lax',
    path: '/',
    maxAge: 60 * 60 * 24 * 7, // 1 week
  });

  return response;
}
}

```

2. Token Management Strategy

Client-Side Token Access: - Stored in localStorage for client-side API calls - Automatically retrieved by OpenAPI client - Used for React Query requests

Server-Side Token Access: - Stored in HTTP-only cookies for security - Used for server-side rendering and API calls - Prevents XSS attacks

Example Server-Side Usage:

```

// In server components
const cookieStore = await cookies();
OpenAPI.TOKEN = cookieStore.get('access_token')?.value;

```

```
const data = await FundsService.getFundsStockByFundIdSummary({
  fundId });
```

3. Security Features

CAPTCHA Integration: - Dynamic CAPTCHA generation - CAPTCHA validation on login - Automatic CAPTCHA refresh on errors

Session Management: - 7-day token expiration - Automatic token refresh (if implemented) - Secure cookie configuration

Error Handling: - Comprehensive error messages - Toast notifications for user feedback - Automatic CAPTCHA refresh on login failures

Rendering Strategies

Next.js 15 App Router Architecture

The project leverages Next.js 15's App Router with sophisticated rendering strategies optimized for different use cases.

1. Server-Side Rendering (SSR)

Implementation Pattern:

```
// Server Component with data fetching
export default async function FundPage() {
  const fundId = 42;
  const cookieStore = await cookies();

  // Set authentication token for server-side API calls
  OpenAPI.TOKEN = cookieStore.get('access_token')?.value;

  // Parallel data fetching
  const [summaryData, analysisData] = await Promise.all([
    FundsService.getFundsStockByFundIdSummary({ fundId }),
    FundsService.getFundsStockByFundIdReturnAnalysis({ fundId }),
  ]);

  return (
    <div>
      <FundTabs summary={summaryData} returnAnalysis={analysisData} />
    </div>
  );
}
```

Benefits: - **SEO Optimization:** Full HTML content available to crawlers - **Fast Initial Load:** Pre-rendered content - **Data Security:** Sensitive API calls on server - **Performance:** Reduced client-side JavaScript

2. Client-Side Rendering (CSR)

Implementation Pattern:

```
'use client';  
// Client Component with React Query  
const InvestmentFundsPage = () => {  
  const {  
    data: fundsTableData,  
    isLoading: isFundsTableLoading,  
    error: fundsTableError,  
  } = useFundsServiceGetFundsTable({  
    tabId: activeIndexCategoryTab,  
    searchQuery: fundSearchQuery,  
  });  
  
  return (  
    <div>  
      {isFundsTableLoading ? <TableSkeleton /> : <FundsTable data=  
        {fundsTableData} />}  
    </div>  
  );  
};
```

Benefits: - **Interactive Features:** Real-time updates, complex state management - **User Experience:** Smooth transitions, optimistic updates - **Dynamic Content:** User-specific data, personalization

3. Hybrid Rendering Strategy

Route-Based Strategy:

Server-Rendered Routes: - /my-fund - Fund details with pre-loaded data - /profile - User profile information - Static content pages

Client-Rendered Routes: - /investment_funds - Interactive table with filtering/sorting - Dashboard pages with real-time updates - Complex form interactions

4. Layout-Based Rendering

Nested Layout Architecture:

```
app/  
├─ layout.tsx                                # Root layout (Client)
```

```

└─ (auth)/
  └─ layout.tsx          # Auth layout (Server)
└─ (dashboard)/
  └─ layout.tsx          # Dashboard layout (Server)
    └─ (withfooter)/
      └─ layout.tsx      # With footer layout
    └─ (nofooter)/
      └─ layout.tsx      # No footer layout

```

Layout Rendering Strategy: - **Root Layout:** Client-side for global providers (React Query, Toast) - **Auth Layout:** Server-side for static authentication UI - **Dashboard Layouts:** Server-side for navigation, client-side for interactive elements

5. Data Fetching Strategies

Server-Side Data Fetching:

```

// Parallel data fetching with Promise.all
const [data1, data2, data3] = await Promise.all([
  FundsService.getEndpoint1(),
  FundsService.getEndpoint2(),
  FundsService.getEndpoint3(),
]);

```

Client-Side Data Fetching:

```

// React Query with caching and background updates
const { data, isLoading, error } = useQuery({
  queryKey: ['funds', fundId],
  queryFn: () => FundsService.getFund(fundId),
  staleTime: 5 * 60 * 1000, // 5 minutes
  gcTime: 0, // No garbage collection time
});

```

6. Performance Optimizations

Code Splitting:

```

// Dynamic imports for heavy components
const HeavyChart = dynamic(() => import('./HeavyChart'), {
  loading: () => <ChartSkeleton />,
  ssr: false, // Client-side only
});

```

Streaming and Suspense:

```

// Streaming server components
export default function Page() {
  return (

```

```
<div>
  <Suspense fallback={<HeaderSkeleton />}>
    <Header />
  </Suspense>
  <Suspense fallback={<ContentSkeleton />}>
    <Content />
  </Suspense>
</div>
);
}
```

Design System - Complete Component Analysis

Overview

The design system is located in `libs/design-system` and provides a comprehensive set of 60+ reusable UI components built with React, TypeScript, and Tailwind CSS.

Component Architecture

Each component follows a consistent structure:

ComponentName/		
├	ComponentName.tsx	# Main component implementation
├	ComponentName.types.ts	# TypeScript interfaces (if multiple)
├	ComponentName.constants.ts	# Component constants (if needed)
├	ComponentName.stories.tsx	# Storybook stories
├	ComponentName.test.tsx	# Unit tests
└	index.ts	# Exports

All Components - Detailed Implementation

Complete Component Inventory

The design system contains **71 components** organized into the following categories:

1. Form & Input Components (12 components)

- **Button** - Multi-variant button with loading states and themes

- **Checkbox** - Checkbox with indeterminate state support
- **TextField** - Advanced text input with validation
- **DateInput** - Date picker with Persian calendar support
- **DatePicker** - Comprehensive date selection component
- **DropDown** - Dropdown with search and multi-select
- **FileUpload** - File upload with drag-and-drop
- **NumberInput** - Numeric input with formatting
- **OptionsDropdown** - Advanced dropdown with filtering
- **Radio** - Radio button groups with custom styling
- **Switch** - Toggle switch component
- **TextArea** - Multi-line text input

2. Data Display Components (15 components)

- **GeneralTable** - Advanced data table with sorting/filtering
- **DataList** - List component for structured data
- **FundsLogo** - Fund logo display with theming
- **FundsTag** - Tag component for fund categorization
- **AssetInfoBox** - Asset information display
- **SummaryCell** - Summary data cell component
- **SummaryCellCarousel** - Carousel of summary cells
- **SparkLine** - Mini chart for trend visualization
- **SlidingNumber** - Animated number transitions
- **Badge** - Status and notification badges
- **BulletList** - Styled bullet point lists
- **ReportCard** - Card for report display
- **ReportCardBase** - Base report card component
- **PerformanceCard** - Performance metrics card
- **NewsCard** - News article card component

3. Navigation & Layout Components (8 components)

- **Breadcrumb** - Navigation breadcrumb trail
- **Tabs** - Tabbed interface component
- **Pagination** - Page navigation component
- **Header** - Application header with navigation
- **Footer** - Application footer component
- **Sidebar** - Collapsible sidebar navigation
- **MobileMenu** - Mobile navigation menu
- **NavigationBar** - Main navigation bar

4. Feedback & Overlay Components (10 components)

- **Dialog** - Modal dialog with focus management
- **ConfirmModal** - Confirmation dialog component
- **BottomSheet** - Mobile bottom sheet modal
- **Tooltip** - Hover tooltip component

- **ToolTipInfo** - Information tooltip variant
- **Toast** - Toast notification system
- **Loading** - Loading spinner and states
- **ProgressBar** - Progress indication component
- **Skeleton** - Loading skeleton components
- **ErrorBoundary** - Error handling component

5. Media & Rich Content Components (6 components)

- **VideoPlayer** - Advanced video player with HLS support
- **ImageGallery** - Image gallery with lightbox
- **Icon** - Icon component with Iconify integration
- **Avatar** - User avatar component
- **Logo** - Application logo component
- **QRCode** - QR code generation component

6. Interactive & Utility Components (12 components)

- **Accordion** - Collapsible content sections
- **ContextMenu** - Right-click context menu
- **SharePopUp** - Social sharing popup
- **FilterPopUpSection** - Advanced filtering interface
- **CommentSection** - Comment system component
- **Bookmark** - Bookmark toggle component
- **AutoRotateSwitch** - Auto-rotation toggle
- **SelectionChips** - Multi-selection chip interface
- **ReportsCarousel** - Carousel for reports
- **AddReportButton** - Report creation button
- **ReportSettings** - Report configuration component
- **SquaredButton** - Square-styled button variant

7. Specialized Components (8 components)

- **FundsColumnHeader** - Table column header for funds
- **LoginForm** - Complete login form component
- **CaptchaInput** - CAPTCHA input component
- **OTPInput** - One-time password input
- **SearchBox** - Advanced search component
- **FilterBar** - Filtering interface bar
- **SectionTitle** - Section heading component
- **StatusIndicator** - Status display component

Detailed Component Analysis

Button Component - Advanced Implementation

Location: `libs/design-system/src/lib/components/Button/Button.tsx`

Interface:

```
export interface ButtonProps extends
  React.ButtonHTMLAttributes<HTMLButtonElement> {
  isLoading?: boolean;
  iconLeft?: IconProps;
  iconRight?: IconProps;
  size?: ButtonSize; // 'sm' | 'md'
  mode?: ButtonMode; // 'primary' | 'secondary' | 'text' |
    'underline'
  align?: 'center' | 'right';
  theme?: 'brand' | 'error' | 'success' | 'neutral';
}
```

Advanced Features:

1. **Multi-Theme Support**: 4 different color themes with consistent state variations
2. **Loading States**: Built-in loading spinner with disabled interaction
3. **Icon Integration**: Left and right icon support with proper spacing
4. **Accessibility**: Full keyboard navigation and ARIA support
5. **State Management**: Hover, active, disabled, and loading states

State Variations:

```
// Primary mode with brand theme
mode === 'primary' && theme === 'brand' && !disabled && !isLoading
  ? 'bg-button-brand-surface-default hover:bg-button-brand-surface-
    hover active:bg-button-brand-surface-pressed'

// Secondary mode with error theme
mode === 'secondary' && theme === 'error' && disabled
  ? 'border-button-error-border-disable text-button-error-label-
    plain-disable border'

// Text mode with underline on hover
mode === 'underline' && !disabled
  ? 'underline-offset-8 transition-transform group-hover:underline'
```

Usage Examples:

```
// Primary button with loading state
<Button mode="primary" theme="brand" isLoading={isSubmitting}>
  Submit Form
</Button>

// Secondary button with icon
<Button mode="secondary" iconLeft={{ name: 'plus' }}>
  Add Item
</Button>
```

```
// Text button with underline effect
<Button mode="underline" theme="neutral">
  Learn More
</Button>
```

TextField Component - Comprehensive Input Solution

Advanced Features: 1. **Real-time Validation:** Custom validation rules with immediate feedback 2. **Debounced Input:** Configurable debounce for performance optimization 3. **Error State Management:** Visual error states with descriptive messages 4. **Accessibility:** Full ARIA support with proper labeling 5. **Custom Styling:** Tailwind-based theming with design tokens

Validation System:

```
interface TextFieldProps {
  validation?: {
    required?: boolean;
    minLength?: number;
    maxLength?: number;
    pattern?: RegExp;
    custom?: (value: string) => string | null;
  };
  debounceMs?: number;
  onValidationChange?: (isValid: boolean) => void;
}
```

Checkbox Component - Advanced Selection

Features: 1. **Indeterminate State:** Three-state checkbox (checked, unchecked, indeterminate) 2. **Group Management:** Checkbox groups with parent-child relationships 3. **Custom Styling:** Design system integration with proper theming 4. **Keyboard Navigation:** Full keyboard accessibility 5. **Form Integration:** Seamless form library integration

Implementation:

```
const Checkbox = ({ indeterminate, checked, onChange, ...props }) => {
  {
    const checkboxRef = useRef<HTMLInputElement>(null);

    useEffect(() => {
      if (checkboxRef.current) {
        checkboxRef.current.indeterminate = indeterminate ?? false;
      }
    }, [indeterminate]);
  }
}
```

```
return (  
  <input  
    ref={checkboxRef}  
    type="checkbox"  
    checked={checked}  
    onChange={onChange}  
    className={cn(  
      'form-checkbox',  
      indeterminate && 'indeterminate-state',  
      checked && 'checked-state'  
    )}  
    {...props}  
  />  
);  
};
```

Complex Components Analysis

1. VideoPlayer Component - Advanced Implementation

Location: `libs/design-system/src/lib/components/VideoPlayer/`

The VideoPlayer is one of the most sophisticated components in the design system, featuring a complete custom video player implementation with advanced controls and state management.

Architecture Overview

The VideoPlayer consists of multiple interconnected parts:

1. **Main VideoPlayer Component** (`VideoPlayer.tsx`)
2. **Custom Hook** (`UseVideo.tsx`) - State management with `useReducer`
3. **Control Panel Subcomponents:**
 - `PlayerActions` - Play/pause, volume controls
 - `PlayerOptions` - Quality, fullscreen, PiP controls
 - `PlayerProgressBar` - Seek bar with thumbnail preview
 - `PlayerThumbnail` - Hover thumbnails on progress bar
 - `VideoTimer` - Current time and duration display

State Management Implementation

The VideoPlayer uses a sophisticated state management system with `useReducer`:

```
interface videoState {
```

```

    isPlaying: boolean;
    currentTime: number;
    duration: number;
    isFinished: boolean;
    progress: number;
    isVideoLoaded: boolean;
    isVideoWaited: boolean; // buffering state
    bufferedTime: number;
    playBackRate: number;
    volume: number;
    muted: boolean;
    isFullscreen: boolean;
    quality: { src: string; label: string; };
    error: string | null;
    src: string;
  }

```

Key State Actions: - PLAY/PAUSE - Control playback state - TIME_UPDATE
 - Track current playback time - SET_VIDEO_LOADED/WAITED - Handle loading and buffering states - SET_QUALITY - Dynamic quality switching
 - SET_FULLSCREEN - Fullscreen mode management

Advanced Features

1. HLS.js Integration

```

import Hls from 'hls.js';
// Supports adaptive streaming for different video qualities

```

2. Control Panel Auto-Hide

```

useEffect(() => {
  let timeout: NodeJS.Timeout;
  const showControls = () => {
    setShowControlPanel(true);
    clearTimeout(timeout);
    timeout = setTimeout(() => setShowControlPanel(false), 5000);
  };
  // Auto-hide controls after 5 seconds of inactivity
}, []);

```

3. Playlist Integration - Fullscreen playlist overlay - Video switching capabilities - Playlist state management - Click-outside-to-close functionality

4. Thumbnail Preview System

```

spriteBaseUrl?: {
  image: string;
  intervalSeconds: number;
}

```

```
};
```

- Hover thumbnails on progress bar
- Sprite-based thumbnail system for performance
- Configurable interval timing

5. Keyboard Navigation Support - Space bar for play/pause - Arrow keys for seeking - Volume controls - Fullscreen toggle

6. Picture-in-Picture Support - Native browser PiP API integration - Fallback handling for unsupported browsers

Performance Optimizations

1. Memoized Components

```
const MemoizedTitle = React.memo(({ title, setShowPlayList }) => {  
  // Prevents unnecessary re-renders of title component  
});
```

2. Event Listener Cleanup

```
useEffect(() => {  
  return () => {  
    videoContainerRef.current?.removeEventListener('mousemove',  
      showControls);  
    clearTimeout(timeout);  
  };  
}, []);
```

3. Conditional Rendering - Controls only render when needed - Playlist overlay only in fullscreen mode - Loading states prevent interaction until ready

Usage Example

```
<VideoPlayer  
  qualities={[  
    { src: 'video-720p.mp4', label: '720p' },  
    { src: 'video-1080p.mp4', label: '1080p' }  
  ]}  
  poster="thumbnail.jpg"  
  title="Investment Fund Analysis"  
  src="video.mp4"  
  spriteBaseUrl={{  
    image: 'thumbnails-sprite.jpg',  
    intervalSeconds: 10  
  }}  
  videos={playlistVideos}  
  selectedVideo={currentVideo}  
  setSelectedVideo={setCurrentVideo}
```

/>

2. GeneralTable Component - Data Management

Location: `libs/design-system/src/lib/components/GeneralTable/`

The GeneralTable is a highly sophisticated data table component built on top of TanStack Table (React Table v8) with advanced features for enterprise data management.

Key Features

1. Column Management - Drag-and-drop column reordering - Column visibility controls - Column resizing - Sticky columns (pinning) - Custom column types

2. Sorting & Filtering - Multi-column sorting - Advanced filtering with multiple operators - Date range filtering - Custom filter components - Filter persistence

3. Data Export - Excel export functionality - PDF export capabilities - Custom export formats - Filtered data export

4. Performance Features - Virtual scrolling for large datasets - Pagination with configurable page sizes - Lazy loading support - Optimized re-rendering

5. User Experience - Row selection (single/multiple) - Row highlighting and marking - Keyboard navigation - Responsive design - Loading states and skeletons

Implementation Details

Column Definition System:

```
interface FundColumnMeta {  
  dragIndicator?: boolean;  
  isSticky?: boolean;  
  stickyPosition?: number;  
  filterType?: 'text' | 'number' | 'date' | 'select';  
  exportable?: boolean;  
}
```

Advanced Sorting: - Server-side sorting integration - Multi-column sort with priority - Custom sort functions - Sort state persistence

Filter System: - Multiple filter types (text, number, date, select) - Range filters for numerical data - Date picker integration - Custom filter components

3. Form Components - Validation & UX

TextField Component

Advanced Features: - Real-time validation - Debounced input handling - Custom validation rules - Error state management - Accessibility compliance (ARIA labels)

```
interface TextFieldProps {
  validation?: {
    required?: boolean;
    minLength?: number;
    maxLength?: number;
    pattern?: RegExp;
    custom?: (value: string) => string | null;
  };
  debounceMs?: number;
  onValidationChange?: (isValid: boolean) => void;
}
```

Checkbox & Radio Components

Features: - Group management - Indeterminate state support - Custom styling with CSS-in-JS - Keyboard navigation - Form integration

4. Dialog/Modal System

Advanced Modal Management: - Modal stacking support - Focus trap implementation - Escape key handling - Backdrop click handling - Animation system - Portal rendering

```
interface DialogProps {
  isOpen: boolean;
  onClose: () => void;
  closeOnBackdrop?: boolean;
  closeOnEscape?: boolean;
  size?: 'sm' | 'md' | 'lg' | 'xl' | 'full';
  animation?: 'fade' | 'slide' | 'scale';
}
```

State Management Patterns

React Query Configuration

Location: apps/fe-app/app/lib/react-query.ts

```

export const queryClient = new QueryClient({
  queryCache: new QueryCache({
    onError: (error) => {
      console.log(error);
    },
  }),
  defaultOptions: {
    queries: {
      retry: false, // No automatic retries
      refetchOnWindowFocus: false, // Don't refetch on window
        focus
      throwOnError: false, // Handle errors gracefully
      gcTime: 0, // No garbage collection time
      staleTime: 5 * 60 * 1000, // 5 minutes stale time
    },
  },
});

```

Provider Setup (apps/fe-app/app/providers/ReactQueryProvider.tsx):

```

export default function ReactQueryProvider({ children }: { children:
  React.ReactNode }) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  );
}

```

State Management Strategies

1. Server State Management (React Query)

Generated Hooks Usage:

```

// Auto-generated React Query hook
const {
  data: fundsTableData,
  isLoading: isFundsTableLoading,
  error: fundsTableError,
  refetch: refetchFundsTable,
} = useFundsServiceGetFundsTable({
  tabId: activeIndexCategoryTab,
  searchQuery: fundSearchQuery,
});

// Mutation for data updates
const updateColumnMutation =
  useFundsServicePostFundsTableTabByTabColumn({

```

```

onSuccess: () => {
  queryClient.invalidateQueries(['fundsTable']);
},
onError: (error) => {
  showToast({ message: 'Update failed', type: 'error' });
},
});

```

2. Client State Management (useState + useReducer)

Complex State with useReducer (VideoPlayer):

```

interface videoState {
  isPlaying: boolean;
  currentTime: number;
  duration: number;
  isFinished: boolean;
  progress: number;
  isVideoLoaded: boolean;
  isVideoWaited: boolean;
  bufferedTime: number;
  playBackRate: number;
  volume: number;
  muted: boolean;
  isFullscreen: boolean;
  quality: { src: string; label: string; };
  error: string | null;
  src: string;
}

const videoReducer = (state: videoState, action: videoAction):
  videoState => {
  switch (action.type) {
    case 'PLAY':
      return { ...state, isPlaying: true, isFinished: false,
        isVideoWaited: false };
    case 'PAUSE':
      return { ...state, isPlaying: false };
    case 'TIME_UPDATE':
      return { ...state, currentTime: action.currentTime };
    // ... more cases
  }
};

```

Page-Level State Management (Investment Funds):

```

const [activeIndexCategoryTab, setActiveIndexCategoryTab] =
  useState(1);
const [isShowDatePicker, setIsShowDatePicker] = useState(false);
const [customColumnNameDate, setCustomColumnNameDate] = useState({

```

```

        start: '', end: '', groupId: ''
    });
    const [isFilterModalOpen, setIsFilterModalOpen] = useState(false);
    const [isSettingModalOpen, setIsSettingModalOpen] = useState(false);
    const [localColumns, setLocalColumns] =
        useState<FundTableTabColumnDto[]>([]);
    const [fundSearchQuery, setFundSearchQuery] = useState<string>('');
    const [pinnedList, setPinnedList] = useState<number[]>([]);
    const [rowsMark, setRowsMark] = useState<{ color: string; id: number
        }[]>([]);

```

3. Form State Management

Login Form State:

```

const FormWrapper = () => {
    const { showToast } = useCustomToast();
    const router = useRouter();
    const refetchCaptchaRef = React.useRef<() => void>(undefined);

    const handleLogin = async (values: LoginFormValues) => {
        try {
            const res = await fetch('/api/login', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    username: values.username,
                    password: values.password,
                    captcha: values.captcha ?? '',
                    captchaUid: typeof values.captchaUid === 'number' ?
                    values.captchaUid : 0,
                }),
            });

            const data = await res.json();
            if (res.ok && data.success) {
                localStorage.setItem('access_token', data.access_token);
                showToast({ message: 'ورود موفقیت آمیز بود', type:
                    'success' });
                router.push('/');
            }
        } catch (error) {
            showToast({ message: 'خطا در ورود', type: 'error'
            });
            if (refetchCaptchaRef.current) {
                refetchCaptchaRef.current();
            }
        }
    };
};

```

Performance Optimization Strategies

1. Component-Level Optimizations

React.memo for Expensive Components:

```
const MemoizedTitle = React.memo(({ title, setShowPlayList }) => {
  return (
    <div className="flex items-center justify-between">
      <h3>{title}</h3>
      <button onClick={() => setShowPlayList(true)}>Show
        Playlist</button>
    </div>
  );
});
```

useMemo for Expensive Calculations:

```
const processedData = useMemo(() => {
  return fundsTableData?.map(item => ({
    ...item,
    formattedValue: formatNumber(item.value),
    calculatedRisk: calculateRisk(item.riskMetrics),
  }));
}, [fundsTableData]);
```

useCallback for Event Handlers:

```
const handleColumnReorder = useCallback((result: DragEndEvent) => {
  if (!result.destination) return;

  const newColumnOrder = arrayMove(
    columnOrder,
    result.source.index,
    result.destination.index
  );

  setColumnOrder(newColumnOrder);
}, [columnOrder]);
```

2. Data Fetching Optimizations

Parallel Data Fetching:

```
const [summaryData, analysisData] = await Promise.all([
  FundsService.getFundsStockByFundIdSummary({ fundId }),
  FundsService.getFundsStockByFundIdReturnAnalysis({ fundId }),
]);
```

Debounced Search:

```
const [searchQuery, setSearchQuery] = useState('');
const debouncedSearchQuery = useDebounce(searchQuery, 300);

useEffect(() => {
  if (debouncedSearchQuery) {
    refetchFundsTable();
  }
}, [debouncedSearchQuery]);
```

Virtual Scrolling for Large Tables:

```
const { scrollElementRef, wrapperElementRef } = useVirtualizer({
  count: tableData.length,
  getScrollElement: () => scrollElementRef.current,
  estimateSize: () => 50,
  overscan: 10,
});
```

3. Bundle Optimization

Dynamic Imports:

```
const HeavyChart = dynamic(() => import('./HeavyChart'), {
  loading: () => <ChartSkeleton />,
  ssr: false,
});

const VideoPlayer = dynamic(() => import('design-system').then(mod
  => ({ default: mod.VideoPlayer })), {
  loading: () => <div>Loading video player...</div>,
});
```

Code Splitting by Route:

```
// Automatic code splitting with Next.js App Router
app/
├─ (dashboard)/
│   └─ investment_funds/page.tsx    # Separate bundle
│   └─ my-fund/page.tsx             # Separate bundle
├─ (auth)/
│   └─ login/page.tsx               # Separate bundle
```

Applications - Every Page Analysis

FE-App (B2B Application) - Detailed Page Breakdown

1. Investment Funds Page (/investment_funds)

Location: apps/fe-

app/app/(dashboard)/(nofooter)/investment_funds/page.tsx

This is the most complex page in the application, serving as the main dashboard for investment fund management.

Implementation Architecture

State Management:

```
const [activeIndexCategoryTab, setActiveIndexCategoryTab] =
  useState(1);
const [isShowDatePicker, setIsShowDatePicker] = useState(false);
const [customColumnDate, setCustomColumnDate] = useState({
  start: '', end: '', groupId: ''
});
const [isFilterModalOpen, setIsFilterModalOpen] = useState(false);
const [isSettingModalOpen, setIsSettingModalOpen] = useState(false);
const [localColumns, setLocalColumns] =
  useState<FundTableTabColumnDto[]>([]);
const [fundSearchQuery, setFundSearchQuery] = useState<string>('');
const [pinnedList, setPinnedList] = useState<number[]>([]);
const [rowsMark, setRowsMark] = useState<{ color: string; id: number
}>([]);
```

Key Features

1. Advanced Table Management - Drag & Drop Columns: Uses @dnd-kit/core for column reordering - **Column Visibility:** Dynamic show/hide columns - **Sticky Columns:** Pin important columns to left/right - **Custom Date Columns:** Add date-range based columns - **Row Marking:** Color-code rows for visual organization - **Pinned Rows:** Pin important funds to top

2. Real-time Data Integration

```
const {
  data: fundsTableData,
  isLoading: isFundsTableLoading,
  error: fundsTableError,
} = useFundsServiceGetFundsTable({
  tabId: activeIndexCategoryTab,
  searchQuery: fundSearchQuery,
});
```

3. Export Functionality - Excel export with custom formatting - PDF generation - Filtered data export - Custom column selection for export

4. Advanced Filtering System - Multi-criteria filtering - Date range filters - Numerical range filters - Text search with debouncing - Filter persistence across sessions

5. Performance Optimizations - Virtual scrolling for large datasets - Smart table scroll with useSmartTableScroll - Debounced search queries - Optimistic updates for user interactions

Component Structure

```
// Main table with drag-and-drop support
<DndContext
  collisionDetection={closestCenter}
  modifiers={[restrictToHorizontalAxis]}
  onDragEnd={handleDragEnd}
  sensors={sensors}
>
  <SortableContext
    items={columnOrder}
    strategy={horizontalListSortingStrategy}
  >
    <GeneralTable
      data={processedData}
      columns={dynamicColumns}
      sorting={sorting}
      onSortingChange={setSorting}
      // ... other props
    />
  </SortableContext>
</DndContext>
```

API Integration

The page integrates with multiple API endpoints: - getFundsTable - Main table data - postFundsTableTabByTabColumn - Column management - postFundsTableTabByTabSort - Sorting preferences - postFundsTableTabByTabColumnsReset - Reset column layout

2. My Fund Page (/my-fund)

Location: apps/fe-app/app/(dashboard)/(withfooter)/my-fund/page.tsx

This page provides detailed analysis of a specific investment fund with comprehensive data visualization.

Server-Side Data Fetching

```
const [
```

```

    { fundBasicInfo, fundSummaryBasicInfo, fundSummaryCaseByCase,
      fundVideoPlaylist },
    { returnComparison, returnRank, returnTrend, riskReturnAnalysis,
      seasonalityEffectAnalysis }
  ] = await Promise.all([
    FundsService.getFundsStockByFundIdSummary({ fundId }),
    FundsService.getFundsStockByFundIdReturnAnalysis({ fundId })
  ]);

```

Key Components

1. Fund Header Section - Fund logo with dynamic theming - Fund name and basic information - Watch list toggle functionality - Breadcrumb navigation

2. Tabbed Interface

```

<FundTabs
  summary={{
    points: fundSummaryCaseByCase!.navHistory.map((item) => ({
      date: item.jdt,
      value: item.revokeNavRials,
    })),
    defaultQuantity: fundSummaryCaseByCase!.navEndOfPeriodRials,
    defaultValueChange:
      fundSummaryCaseByCase!.returnEndOfPeriodRials,
    defaultPercentageChange:
      fundSummaryCaseByCase!.returnEndOfPeriodPercent,
    fundSummaryBasicInfo: fundSummaryBasicInfo!,
    fundSummaryCaseByCase: fundSummaryCaseByCase!,
    fundVideoPlaylist: fundVideoPlaylist!,
  }}
  returnAnalysis={{
    returnComparison, returnRank, returnTrend,
    riskReturnAnalysis, seasonalityEffectAnalysis,
  }}
/>

```

3. Data Visualization Components - **LineChart**: NAV history visualization - **BubbleChart**: Risk-return analysis - **ChangeComparisonFunds**: Comparative analysis - **VideoPlayer Integration**: Educational content

Tab Structure

Summary Tab (Summary.tsx): - NAV history chart with interactive tooltips - Key performance metrics - Fund composition breakdown - Recent performance indicators

Return Analysis Tab (Return.tsx): - Comparative return analysis - Benchmark comparisons - Historical performance trends - Risk-adjusted returns

Risk Assessment Tab (Risk.tsx): - Risk metrics visualization - Volatility analysis - Drawdown analysis - Risk-return scatter plots

3. Profile Page (/profile)

Location: apps/fe-app/app/(dashboard)/(withfooter)/profile/page.tsx

User profile management with comprehensive settings and preferences.

Features

- Personal information management
- Investment preferences
- Notification settings
- Security settings
- Account verification status

4. Reports System - Complete Implementation

Reports List Page (apps/fe-app/app/(dashboard)/(withfooter)/reports/page.tsx):

Advanced Server-Side Architecture:

Data Fetching Strategy:

```
async function getData(searchParams: GetReportsData) {  
  // Server-side authentication  
  const cookieStore = await cookies();  
  OpenAPI.TOKEN = cookieStore.get('access_token')?.value;  
  
  // Parallel API calls for performance  
  const [reports, categories] = await Promise.all([  
    ReportsService.getReports({ ...searchParams }),  
    ReportsService.getReportsCategories(),  
  ]);  
  return { reports, categories };  
}
```

Advanced Filtering System:

```
const filteredReports = reports.filter((report) => {  
  const matchesCategory = resolvedSearchParams.category  
    ? report.category.title === resolvedSearchParams.category  
    : true;  
});
```

```

const matchesSearch = resolvedSearchParams.search
  ? report.title.includes(resolvedSearchParams.search)
  : true;

return matchesCategory && matchesSearch;
});

```

Pagination Implementation:

```

const ITEMS_PER_PAGE = 6;
const currentPage = resolvedSearchParams.page ?
  parseInt(resolvedSearchParams.page, 10) : 1;
const totalPages = Math.ceil(filteredReports.length /
  ITEMS_PER_PAGE);

const paginatedReports = filteredReports.slice(
  (currentPage - 1) * ITEMS_PER_PAGE,
  currentPage * ITEMS_PER_PAGE,
);

```

Component Architecture: - **ReportList:** Main report grid with infinite scroll support - **SideBar:** Category navigation and filtering - **NewReportDialog:** Report creation modal - **SearchBar:** Real-time search functionality - **FilterReport:** Advanced filtering options

Individual Report Page (apps/fe-app/app/(dashboard)/(withfooter)/report/[report_id]/page.tsx):

Comprehensive Report Display System:

Advanced Metadata Generation:

```

export async function generateMetadata({ params, searchParams }:
  ReportPageParams): Promise<Metadata> {
  const { report_id: id } = await params;
  const cookieStore = await cookies();
  OpenAPI.TOKEN = cookieStore.get('access_token')?.value;

  const REPORT = await getData(id, screenshotQueryId);
  const baseUrl = process.env.NEXT_PUBLIC_API_URL;

  const imageUrl = REPORT.screenshotUrl
    ? `${baseUrl}${REPORT.screenshotUrl}`
    : REPORT.image ? `${baseUrl}${REPORT.image}` : null;

  return {
    title: REPORT.title,
    description: REPORT.summary,
    openGraph: {

```

```

        title: REPORT.title,
        description: REPORT.summary,
        images: imageUrl ? [{ url: imageUrl, width: 1200, height: 630,
            alt: REPORT.title }] : [],
        type: 'article',
    },
    twitter: {
        card: 'summary_large_image',
        title: REPORT.title,
        description: REPORT.summary,
        images: imageUrl ? [imageUrl] : [],
    },
};
}

```

Report Structure:

1. **Breadcrumb Navigation:** Contextual navigation back to reports list
2. **Report Overview Section:** Title, category, summary, favorite toggle
3. **Dynamic Report Renderer:** Interactive financial calculations
4. **Video Analysis Section:** Educational video content
5. **Markdown Content:** Detailed analysis and insights
6. **Related Reports Carousel:** Contextual report recommendations

Dynamic Report Rendering:

```

<ReportWrapper
  data={REPORT.reportCalculation}
  title={REPORT.title}
  identifier={REPORT.identifier}
/>

```

Report Wrapper Implementation (ReportWrapper.tsx):

```

export const ReportWrapper: React.FC<ReportWrapperProps> = ({ data,
  identifier, title }) => {
  const [reportData, setReportData] =
    useState<FinancialReportCalculationApiModel | null>(data);
  const { mutateAsync } = useReportsServicePostReportsById();

  const handleSubmit = async (changedOptions: Record<string,
    OptionItem>) => {
    try {
      const updatedReport = await mutateAsync({
        reportId: String(identifier),
        requestBody: {
          selectedFilters: Object.fromEntries(
            Object.entries(changedOptions).map(([key, { id }]) =>
              [key, String(id)]),
          ),

```

```

        },
    });
    setReportData(updatedReport);
    return true;
  } catch (error) {
    console.error('Error submitting report update', error);
    return false;
  }
};

return (
  reportData && (
    <DynamicReportRenderer
      identifier={identifier}
      data={reportData?.calculation}
      filters={reportData?.filters}
      onSubmit={(changed) => handleSubmit(changed)}
      title={title}
    />
  )
);
};

```

Advanced Features:

1. **Interactive Calculations:** Real-time report parameter adjustments
2. **Favorite System:** User can bookmark reports for quick access
3. **Video Integration:** Educational content with VideoPlayer component
4. **Markdown Rendering:** Rich text content with proper formatting
5. **Related Reports:** AI-powered content recommendations
6. **Social Sharing:** Report sharing capabilities
7. **Export Options:** PDF and other format exports

Report List Component (ReportsList.tsx):

Advanced Features: - **Infinite Scroll:** Automatic loading of more reports - **Favorite Management:** Add/remove favorites with optimistic updates - **Empty States:** Elegant handling of no results - **Responsive Grid:** Adaptive layout for different screen sizes - **Image Optimization:** Dynamic image loading with fallbacks

Favorite Management:

```

const addFavoriteMutation =
  useReportsServicePostReportsByIdFavorite();
const deleteFavoriteMutation =
  useReportsServiceDeleteReportsByIdFavorite();

```

```
const handleLike = async (reportId: string, isFavorite: boolean) =>
{
  if (isFavorite) {
    deleteFavoriteMutation.mutate({ reportId });
  } else {
    addFavoriteMutation.mutate({ reportId: String(reportId) });
  }
};
```

Infinite Scroll Implementation:

```
const handleIntersection = useCallback((entries:
  IntersectionObserverEntry[]) => {
  const [entry] = entries;
  if (entry.isIntersecting && hasMore && onLoadMore) {
    onLoadMore();
  }
}, [hasMore, onLoadMore]);

useEffect(() => {
  const observer = new IntersectionObserver(handleIntersection, {
    threshold: 0.1 });
  if (loadMoreRef.current) {
    observer.observe(loadMoreRef.current);
  }
  return () => {
    if (loadMoreRef.current) {
      observer.unobserve(loadMoreRef.current);
    }
  };
}, [handleIntersection]);
```

Report Categories: - **Portfolio Performance:** Comprehensive portfolio analysis - **Fund Analysis:** Individual fund deep-dives - **Risk Assessment:** Risk metrics and analysis - **Market Analysis:** Market trends and insights - **Comparative Studies:** Fund and portfolio comparisons - **Custom Reports:** User-generated analysis

Export Capabilities: - **PDF Generation:** Professional report formatting - **Excel Export:** Data tables and charts - **Email Delivery:** Automated report distribution - **Scheduled Reports:** Recurring report generation

B2C-App (Consumer Application) - Complete Page Analysis

1. Authentication Flow Pages - Detailed Implementation

AuthLanding Page (apps/b2c-app/app/(auth)/AuthLanding/page.tsx):

Purpose: Consumer onboarding with sophisticated animated carousel showcasing Aress Investor benefits.

Advanced Features: - **5-Slide Lottie Animation Carousel:** Each slide features professional Lottie animations - **Automatic**

Progression: 4.5-second intervals with smooth fade transitions -

Interactive Indicators: Clickable dot navigation for manual slide control -

Responsive Design: Optimized for mobile and desktop experiences

Slide Content Strategy:

```
const slidesData = [
  {
    title: 'پردازش',
    'اطلاعات مالی آرسس',
    description: 'با آرسس',
    'اینوستور، سرمایه گذاری آسان و',
    'آینده ای روشن در دسترس شماست',
    lottieAnimation: SuccessfulMarketerAnimation,
  },
  {
    title: 'زیر نظر',
    'سازمان بورس',
    description: 'ما در آرسس',
    'اینوستور بستری امن برای',
    'سرمایه گذاری آسان شما فراهم کردیم',
    lottieAnimation: OnlineBankingAnimation,
  },
  // ... 3 more slides
];
```

State Management:

```
const [currentIndex, setCurrentIndex] = useState(0);

useEffect(() => {
  const interval = setInterval(() => {
    setCurrentIndex((prev) => prev === slidesData.length - 1 ? 0 :
      prev + 1);
  }, 4500);
  return () => clearInterval(interval);
}, [currentIndex]);
```

Navigation Strategy: - **Primary CTA:** “ورود” (Login) - Primary button leading to Signin - **Secondary CTA:** “ثبت نام” (Register) - Secondary button leading to Signup - **Fund Manager Access:** Underline button for Aress Terminal access

Signin Page (apps/b2c-app/app/(auth)/Signin/page.tsx):

Multi-Step Authentication Process:

Step 1: National ID Verification

```
const NationalIdForm = ({ onSubmit, type }) => {
  const { control, handleSubmit, formState: { isSubmitting } } =
    useForm<NationalIdFormValues>({
      defaultValues: { nationalCode: '' },
    });

  // National code validation with custom validator
  rules={{
    required: { value: true, message: 'این فیلد اجباری است' },
    validate: (value) => {
      if (!validateNationalCode(value)) {
        return 'لطفا یک کد ملی معتبر وارد کنید';
      }
      return true;
    },
  }}
};
```

Step 2: OTP Verification - SMS Integration: Connects with Sejam system for OTP delivery - **Countdown Timer:** 120-second countdown with resend functionality - **Error Handling:** Comprehensive error states and retry mechanisms

Progress Bar Integration:

```
const PROGRESS_BAR_ITEMS: ProgressBarItemType[] = [
  { text: 'کد ملی', status: 'success' },
  { text: 'رمز یکبار مصرف', status: 'success' },
  { text: 'ورود', status: 'success' },
];
```

Signup Page (apps/b2c-app/app/(auth)/Signup/page.tsx):

Enhanced Registration Flow:

Step 1-2: Same as Signin (National ID + OTP) **Step 3:** Registration Completion with Success Result

Success Result Component:

```
{currentStep === 0 || currentStep === 1 ? (
  <AuthForm
    currentStep={currentStep}
    onNextStep={handleNextStep}
    onPrevStep={handlePrevStep}
    type="signup"
  />
```

```
    />
  ) : (
    <Result type="success" />
  )}
```

Terms and Conditions Integration:

```
<Checkbox
  onChange={() => setConfirmedRules((prev) => !prev)}
  checked={confirmedRules}
  reactcontent={
    <div className="flex flex-row gap-2 font-medium">
      <span className="text-text-brand-primary-600">قوانین و مقررات</span>
      <span className="text-text-neutral-primary">آرئس ائنوسئور را میپذیرم</span>
    </div>
  }
/>
```

2. Portfolio Management - Advanced Implementation

My Portfolio Page (apps/b2c-app/app/(withfooter)/my-portfolio/_components/MyPortfolioPage.tsx):

Comprehensive Portfolio Management System:

State Management Architecture:

```
// Chart state
const [hoveredData, setHoveredData] = useState<HoverData | null>(null);
const [hiddenContent, setHiddenContent] = useState(false);

// Tabs state
const [chartActiveTab, setChartActiveTab] = useState(0);
const [fundsActiveTab, setFundsActiveTab] = useState(0);

// Fund selection state
const [selectedFundType, setSelectedFundType] = useState<number | null>(null);
const [isDetailView, setIsDetailView] = useState(false);
const [selectedFund, setSelectedFund] = useState<FundData | null>(null);

// Modal state
const [showEmptyModal, setShowEmptyModal] = useState(false);
const [emptyCategory, setEmptyCategory] = useState<number | null>(null);

// Trade popup state
const [showTradePopup, setShowTradePopup] = useState(false);
const [tradeMode, setTradeMode] = useState<'buy' | 'sell'>('buy');
```

Advanced Features:

1. Interactive Chart System:

- **Hover Data Display:** Real-time value updates on chart hover
- **Hidden Content Toggle:** Privacy mode for sensitive financial data
- **Multi-timeframe Tabs:** 1M, 3M, 6M, 9M, 1Y chart periods

2. Dynamic Fund Visualization:

- **Category View:** High-level fund type breakdown (سهامی، درآمد، مختلط، کالایی)
- **Detail View:** Individual fund breakdown within categories
- **Empty State Handling:** Modal prompts for empty categories

3. Pie Chart Integration:

```
const currentPieChartData = useMemo(() => {
  if (isDetailView && selectedFundType !== null) {
    // Show individual funds in the selected category
    const categoryFunds = detailedFundsData[selectedFundType] || [];
    return categoryFunds
      .filter((fund) => fund.dailyValue > 0)
      .map((fund) => ({
        name: fund.name || 'نا مشخص',
        value: fund.dailyValue,
      }));
  } else {
    // Show category-level data
    return categoryFundsData
      .filter((fund) => fund.dailyValue > 0)
      .map((fund) => ({
        name: categoryNames[fund.typeID] || 'نا مشخص',
        value: fund.dailyValue,
      }));
  }
}, [isDetailView, selectedFundType]);
```

4. Trading Interface:

- **Buy/Sell Popup:** Modal interface for fund transactions
- **Fund Selection:** Sticky bottom bar for fund selection
- **Price Estimation:** Real-time price and unit calculations

5. Fund Type Management:

```
const fundTypeMaps: Record<number, { title: string; theme: string }>
  = {
    0: { title: 'سهامی', theme: 'green' },
    1: { title: 'درآمد ثابت', theme: 'blue' },
    2: { title: 'مختلط', theme: 'purple' },
    3: { title: 'کالایی', theme: 'yellow' },
  };
```

Data Flow Architecture: - **Mock Data Integration:** Comprehensive fund data with realistic values - **Category Navigation:** Seamless switching between category and detail views - **Empty State Management:** Intelligent handling of empty fund categories

3. Profile Management - Complete System

Profile Layout (apps/b2c-app/app/(withfooter)/(profile)/layout.tsx):

Sophisticated Navigation System: - **Sidebar Navigation:** Profile, Active Sessions, Login History, Messages - **Responsive Design:** Mobile-optimized navigation patterns - **Active State Management:** Visual indicators for current page

Profile Page (apps/b2c-app/app/(withfooter)/(profile)/profile/page.tsx):

Three-Component Architecture:

```
export default function Profile() {
  return (
    <div className="block max-w-[1032px] flex-grow">
      <ProfileForm />           // Personal information management
      <AddressForm />          // Address and location data
      <BankAccountInformation /> // Financial account details
    </div>
  );
}
```

Security Pages: - **Active Sessions:** Real-time session monitoring and management - **Login History:** Comprehensive audit trail of account access - **Messages:** Notification center for account communications

4. Additional B2C Features

FAQ Page (apps/b2c-app/app/(withfooter)/faqs/page.tsx): -

Comprehensive Help System: Categorized frequently asked questions - **Search Functionality:** Quick access to relevant help topics - **Interactive Accordion:** Expandable Q&A sections

Me Page (apps/b2c-app/app/(withfooter)/me/page.tsx): - **Personal Dashboard:** Quick access to account overview - **Shortcut Navigation:** Fast links to common actions - **Account Status:** Real-time account health indicators

Development Setup

Prerequisites

- **Node.js:** Version 18+ (LTS recommended)
- **pnpm:** Version 8+ (package manager)
- **Nx CLI:** Global installation recommended

Installation Steps

1. Install Global Dependencies:

```
npm install -g nx pnpm
```

2. Clone and Setup:

```
git clone <repository-url>  
cd aress-frontend  
pnpm install
```

3. Environment Configuration:

```
# Copy environment templates  
cp apps/fe-app/.env.local.example apps/fe-app/.env.local  
# Configure your environment variables
```

Development Commands

Application Development

```
# Run main B2B application  
nx dev fe-app
```

```
# Run B2C application  
nx dev b2c-app
```

```
# Run both applications  
nx run-many --target=dev --projects=fe-app,b2c-app
```

Design System Development

```
# Run Storybook for component development  
nx storybook design-system
```

```
# Build design system  
nx build design-system
```

API Integration

OpenAPI/Swagger Integration

The project uses a sophisticated API integration strategy with automatic code generation:

Workflow

1. **API Specification:** OpenAPI/Swagger definitions
2. **Code Generation:** Automatic TypeScript client generation
3. **React Query Integration:** Generated hooks for data fetching
4. **Type Safety:** Full TypeScript support

Commands

Merge multiple Swagger files

pnpm merge:swagger

Generate TypeScript API client

pnpm generate:sdk

Generate React Query hooks

pnpm generate:query

Complete API update workflow

pnpm output:swagger

Generated Structure

libs/openapi/src/

— queries/	# React Query hooks
— requests/	# API request functions
— types/	# TypeScript interfaces

Testing Strategy

Testing Pyramid

Unit Tests (Vitest)

- **Component Testing:** Individual component behavior
- **Utility Testing:** Helper function validation
- **Hook Testing:** Custom hook functionality

Integration Tests (Jest)

- **API Integration:** Service layer testing

- **Component Integration:** Multi-component workflows
- **State Management:** Complex state scenarios

E2E Tests (Playwright)

- **User Journeys:** Critical application flows
 - **Cross-browser Testing:** Chrome, Firefox, Safari
 - **Mobile Testing:** Responsive behavior validation
-

Deployment

Build Process

Production Builds

Build all applications

```
nx build
```

Build specific application

```
nx build fe-app
```

```
nx build b2c-app
```

Build design system

```
nx build design-system
```

Docker Deployment

Multi-stage Docker builds

```
docker build -f Dockerfile.fe-app -t aress-fe-app:latest .
```

```
docker build -f Dockerfile.b2c-app -t aress-b2c-app:latest .
```

Deployment Targets

Vercel Integration

- **Automatic Deployments:** Git-based deployments
- **Preview Deployments:** PR-based previews
- **Environment Variables:** Secure configuration management

Container Orchestration

- **Docker Compose:** Local development
 - **Kubernetes:** Production orchestration
 - **Load Balancing:** High availability setup
-

Best Practices & Guidelines

Component Development

1. **Single Responsibility:** Each component has one clear purpose
2. **Prop Interface:** Well-defined TypeScript interfaces
3. **Default Props:** Sensible defaults for optional props
4. **Error Boundaries:** Graceful error handling
5. **Accessibility:** WCAG 2.1 compliance

Performance Optimization

1. **Code Splitting:** Route-based and component-based splitting
2. **Lazy Loading:** Dynamic imports for heavy components
3. **Memoization:** React.memo for expensive components
4. **Bundle Analysis:** Regular bundle size monitoring

Security Considerations

1. **Input Validation:** Client and server-side validation
 2. **XSS Prevention:** Proper data sanitization
 3. **CSRF Protection:** Token-based protection
 4. **Dependency Scanning:** Regular security audits
-

Troubleshooting

Common Issues

Build Errors

- **TypeScript Errors:** Check type definitions and imports
- **Missing Dependencies:** Verify package.json and node_modules
- **Path Resolution:** Check tsconfig.json path mappings

Development Issues

- **Hot Reload:** Restart development server
 - **Port Conflicts:** Check for running processes
 - **Cache Issues:** Clear Nx cache with `nx reset`
-

This comprehensive documentation provides detailed implementation insights for developers working with the Aress Frontend Project. For the most current information, please refer to the project repository and Storybook documentation.

